Ex. No. 5	**Application State Management**
Date of Exercise	
Github Code Link (Keep as public)	
Video Link (Google Drive Link)	

Aim

Develop a Personal Budget Calculator that accepts user income and expenses, calculates the remaining balance, and displays the financial status using Flutter state management techniques.

Description

Every Flutter widget that needs to remember and react to changing data over time is backed by some form of state management. The simplest and most fundamental technique is the built-in StatefulWidget together with its setState() method: whenever setState() is called inside a State object, Flutter marks that widget dirty and rebuilds it (and its descendants) with the new values on the next frame. For a single self-contained screen like a budget calculator, where the state (income, expenses and the computed result) is only ever read and written within that one screen, setState is the appropriate and idiomatic technique — reaching for an external package such as Provider, Riverpod or Bloc would add indirection without any real benefit, since there is no state to share across distant widgets or screens.

This experiment implements a Personal Budget Calculator. A Form containing two TextFormFields (Monthly Income and Total Expenses) uses a validator function to reject empty, non-numeric or negative input. A BudgetResult model wraps the two numbers and exposes a computed balance (income - expenses) and an isSurplus flag. Pressing the Calculate button validates the form and calls setState to store a new BudgetResult, which rebuilds the screen and reveals a BudgetStatusCard: a green card with an upward trend icon when the balance is zero or positive ('You're in surplus'), or a red card with a downward trend icon when it is negative ('You're in deficit'), along with the income, expenses and remaining balance.

Program

lib/models/budget_result.dart

```
class BudgetResult {
  final double income;
  final double expenses;

  const BudgetResult({required this.income, required this.expenses});
```

```
  double get balance => income - expenses;

  bool get isSurplus => balance >= 0;
}
```

lib/widgets/budget_status_card.dart

```
import 'package:flutter/material.dart';

import '../models/budget_result.dart';

class BudgetStatusCard extends StatelessWidget {
  final BudgetResult result;

  const BudgetStatusCard({super.key, required this.result});

  @override
  Widget build(BuildContext context) {
    final bool surplus = result.isSurplus;
    final Color color = surplus ? Colors.green : Colors.red;

    return Card(
      color: color.withValues(alpha: 0.1),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(16),
        side: BorderSide(color: color, width: 1.5),
      ),
      child: Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Icon(surplus ? Icons.trending_up : Icons.trending_down, color:
color),
                const SizedBox(width: 8),
                Text(
                  surplus ? "You're in surplus" : "You're in deficit",
                  style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold,
color: color),
                ),
              ],
            ),
            const SizedBox(height: 12),
            Text(
              'Remaining balance: ₹${result.balance.toStringAsFixed(2)}',
              style: const TextStyle(fontSize: 16),
            ),
            const SizedBox(height: 4),
            Text('Income: ₹${result.income.toStringAsFixed(2)}',
                style: TextStyle(color: Colors.grey.shade700)),
            Text('Expenses: ₹${result.expenses.toStringAsFixed(2)}',
                style: TextStyle(color: Colors.grey.shade700)),
          ],
        ),
      ),
    );
  }
}
```

lib/screens/budget_calculator_screen.dart

```
import 'package:flutter/material.dart';

import '../models/budget_result.dart';
import '../widgets/budget_status_card.dart';

class BudgetCalculatorScreen extends StatefulWidget {
```

```
  const BudgetCalculatorScreen({super.key});

  @override
  State<BudgetCalculatorScreen> createState() =>
_BudgetCalculatorScreenState();
}

class _BudgetCalculatorScreenState extends State<BudgetCalculatorScreen> {
  final _formKey = GlobalKey<FormState>();
  final _incomeController = TextEditingController();
  final _expensesController = TextEditingController();

  BudgetResult? _result;

  @override
  void dispose() {
    _incomeController.dispose();
    _expensesController.dispose();
    super.dispose();
  }

  String? _validateAmount(String? value) {
    if (value == null || value.trim().isEmpty) return 'Required';
    final parsed = double.tryParse(value);
    if (parsed == null) return 'Enter a valid number';
    if (parsed < 0) return 'Must be zero or positive';
    return null;
  }

  void _calculate() {
    if (!_formKey.currentState!.validate()) return;
    setState(() {
      _result = BudgetResult(
        income: double.parse(_incomeController.text),
        expenses: double.parse(_expensesController.text),
      );
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Personal Budget Calculator')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              TextFormField(
                controller: _incomeController,
                keyboardType: const TextInputType.numberWithOptions(decimal:
true),
                decoration: const InputDecoration(
                  labelText: 'Monthly Income (₹)',
                  border: OutlineInputBorder(),
                  prefixIcon: Icon(Icons.account_balance_wallet_outlined),
                ),
                validator: _validateAmount,
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: _expensesController,
                keyboardType: const TextInputType.numberWithOptions(decimal:
true),
                decoration: const InputDecoration(
                  labelText: 'Total Expenses (₹)',
                  border: OutlineInputBorder(),
                  prefixIcon: Icon(Icons.receipt_long_outlined),
                ),
                validator: _validateAmount,
```

```
              ),
              const SizedBox(height: 20),
              ElevatedButton.icon(
                onPressed: _calculate,
                icon: const Icon(Icons.calculate_outlined),
                label: const Padding(
                  padding: EdgeInsets.symmetric(vertical: 12),
                  child: Text('Calculate', style: TextStyle(fontSize: 16)),
                ),
              ),
              const SizedBox(height: 24),
              if (_result != null) BudgetStatusCard(result: _result!),
            ],
          ),
        ),
      ),
    );
  }
}
```

lib/main.dart

```
import 'package:flutter/material.dart';

import 'screens/budget_calculator_screen.dart';

void main() {
  runApp(const BudgetApp());
}

class BudgetApp extends StatelessWidget {
  const BudgetApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Personal Budget Calculator',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        useMaterial3: true,
      ),
      home: const BudgetCalculatorScreen(),
    );
  }
}
```

Output Screenshots

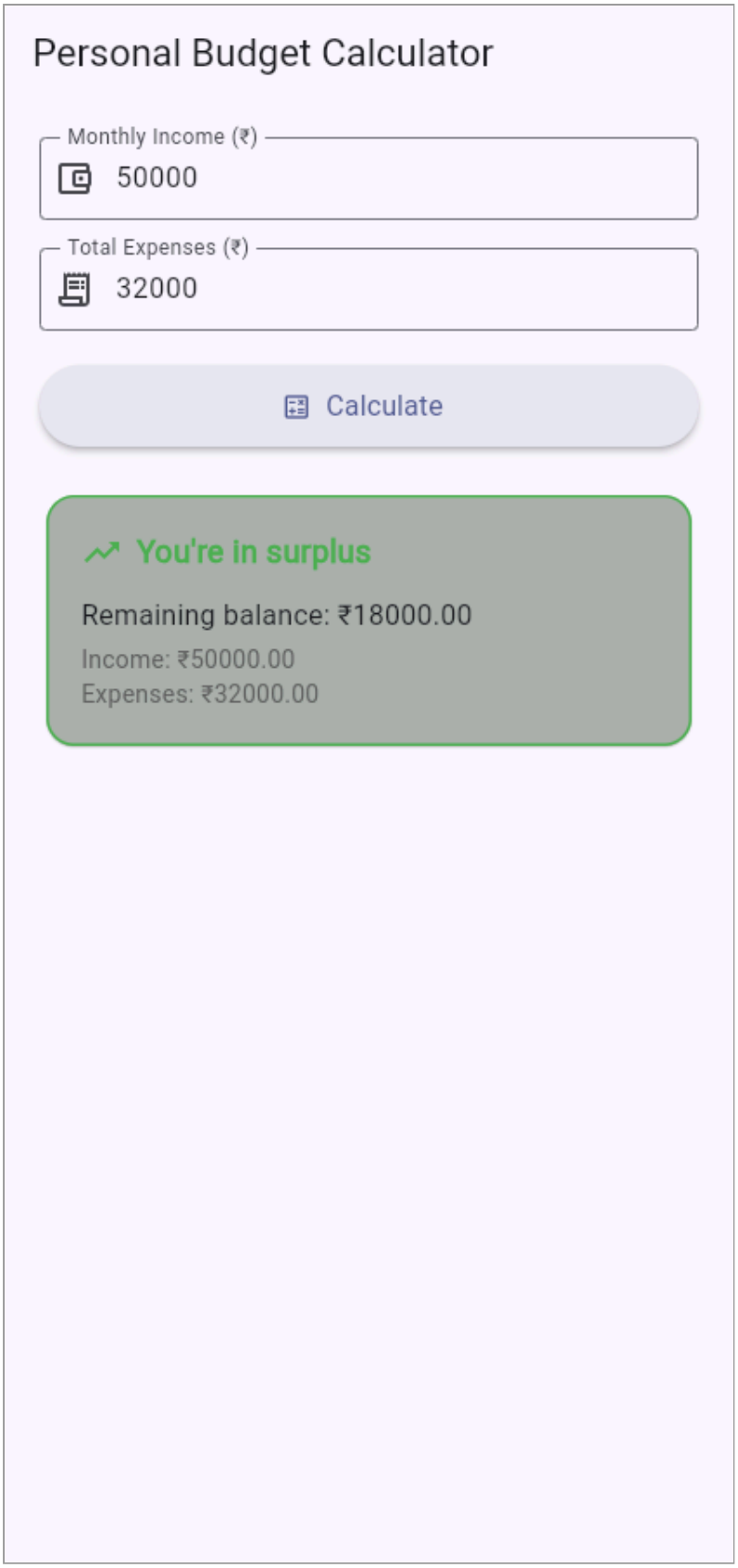


Fig 1. Income 50000, Expenses 32000 — setState rebuilds the screen with a green surplus card showing a remaining balance of 18000.

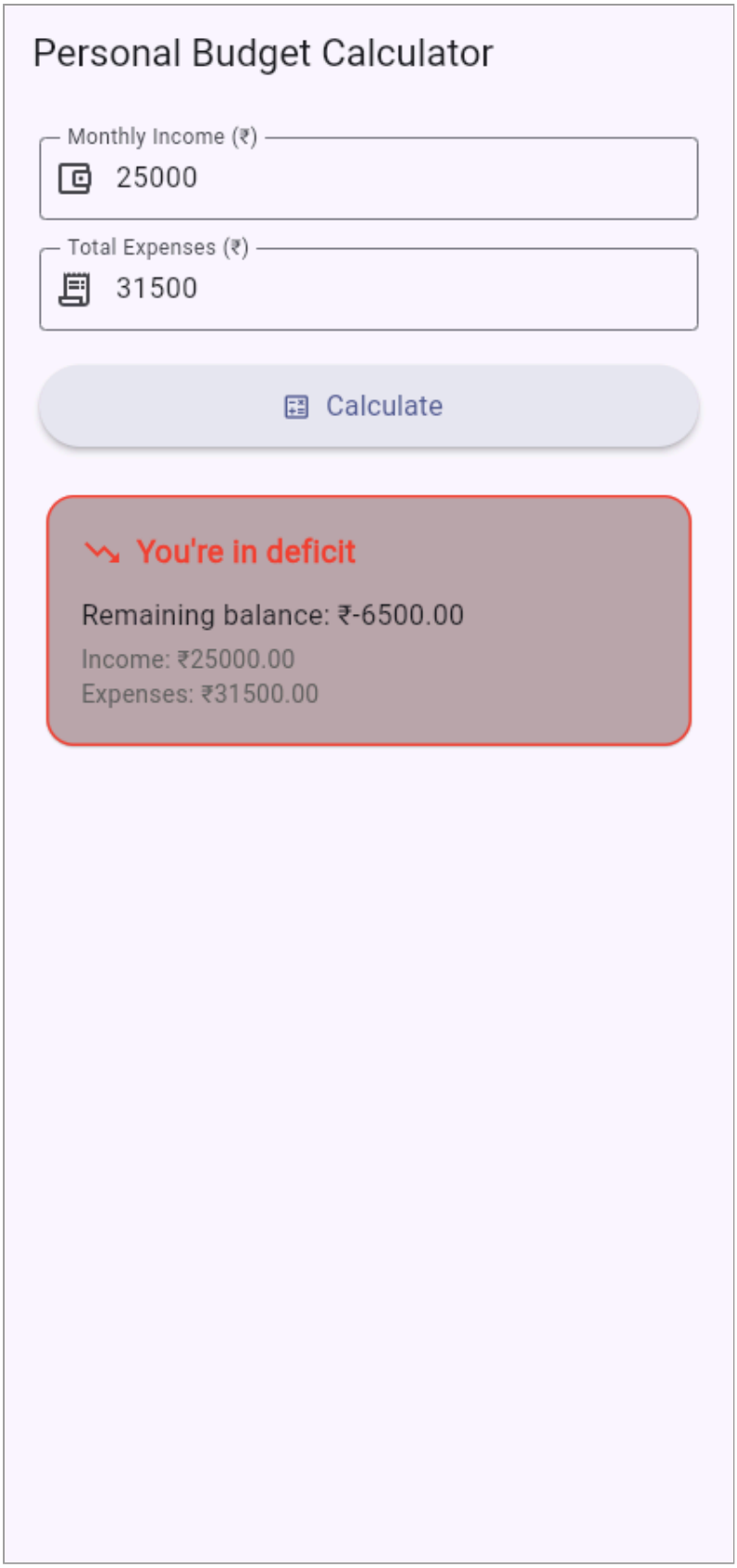


Fig 2. Income 25000, Expenses 31500 — setState rebuilds the screen with a red deficit card showing a remaining balance of -6500.

Result

The Personal Budget Calculator was successfully designed and executed. User-entered income and expenses were validated, the remaining balance was calculated, and the financial status (surplus/deficit) was displayed dynamically using Flutter's built-in setState-based state management, as verified in the screenshots above.